

Jiffies in Linux Kernel

This is a copy of a shared ChatGPT conversation

What are jiffies in Kernel ? How they are helpful ? Provide an example driver code and user app for the jiffies. List the use cases

Jiffies in Linux Kernel

What are Jiffies?

In the Linux kernel, **jiffies** are a global counter used to measure **time**.

- The variable **jiffies** increments at every **timer interrupt**.
- One increment = one **tick** of the kernel timer.
- The frequency of increment depends on the kernel configuration value:

```
C
HZ
```

Example:

HZ Value	Meaning
100	100 ticks/sec
250	250 ticks/sec
1000	1000 ticks/sec

If:

```
C
HZ = 1000
```

then:

```
1 jiffy = 1 millisecond
```

Where is it Defined?

```
C
#include <linux/jiffies.h>
```

Kernel provides:

```
C
extern unsigned long volatile jiffies;
```

Why are Jiffies Helpful?

Jiffies provide a lightweight mechanism for:

- Measuring elapsed time
- Creating delays
- Scheduling timeout operations
- Polling hardware
- Detecting inactivity
- Driver timing operations

Since kernel code cannot always use standard library timing functions, jiffies become the basic kernel timing mechanism.

Important Macros

1. Reading Current Jiffies

```
C
unsigned long now = jiffies;
```

2. Convert Seconds to Jiffies

```
C
msecs_to_jiffies(ms)
secs_to_jiffies(sec)
usecs_to_jiffies(us)
```

Example:

```
C
timeout = jiffies + msecs_to_jiffies(500);
```

3. Time Comparison Macros

Never compare directly like:

```
C
if (jiffies > timeout)
```

Use:

```
C
```

```
time_after(a, b)
time_before(a, b)
```

Because jiffies can overflow.

Example:

```
C
if (time_after(jiffies, timeout))
```

Jiffies Overflow

`jiffies` is an unsigned long.

Eventually it wraps around:

```
4294967295 → 0
```

Linux macros safely handle this overflow.

Simple Character Driver Using Jiffies

This driver:

- Stores load time
- Calculates elapsed seconds
- Sends elapsed time to user application

Kernel Driver Code

```
C
// jiffies_driver.c

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/jiffies.h>

#define DEVICE_NAME "jiffies_dev"

static int major;
static unsigned long start_jiffies;

static ssize_t driver_read(struct file *file,
                          char __user *buffer,
                          size_t len,
                          loff_t *offset)
{
    char msg[100];
    int bytes;
    unsigned long elapsed;

    if (*offset > 0)
        return 0;
}
```

```

elapsed = jiffies - start_jiffies;

bytes = sprintf(msg,
                "Elapsed Jiffies: %lu\nElapsed Seconds: %lu\n",
                elapsed,
                elapsed / HZ);

copy_to_user(buffer, msg, bytes);

*offset = bytes;

return bytes;
}

static struct file_operations fops =
{
    .owner = THIS_MODULE,
    .read = driver_read,
};

static int __init jiffies_init(void)
{
    start_jiffies = jiffies;

    major = register_chrdev(0, DEVICE_NAME, &fops);

    printk(KERN_INFO "Jiffies Driver Loaded\n");
    printk(KERN_INFO "Start Jiffies = %lu\n", start_jiffies);

    return 0;
}

static void __exit jiffies_exit(void)
{
    printk(KERN_INFO "Jiffies Driver Removed\n");

    unregister_chrdev(major, DEVICE_NAME);
}

module_init(jiffies_init);
module_exit(jiffies_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("OpenAI");
MODULE_DESCRIPTION("Simple Jiffies Driver");

```

Makefile

```

Makefile
obj-m += jiffies_driver.o

KDIR = /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean

```

Compile Driver

```

Bash
make

```

Insert Module

```
Bash
sudo insmod jiffies_driver.ko
```

Check Kernel Logs

```
Bash
dmesg
```

Example:

```
Jiffies Driver Loaded
Start Jiffies = 429201234
```

Create Device File

Check major number:

```
Bash
cat /proc/devices
```

Create node:

```
Bash
sudo mknod /dev/jiffies_dev c <major> 0
```

Example:

```
Bash
sudo mknod /dev/jiffies_dev c 240 0
```

User Application

```
C
// app.c

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char buffer[100];

    fd = open("/dev/jiffies_dev", O_RDONLY);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    read(fd, buffer, sizeof(buffer));

    printf("%s\n", buffer);
}
```

```
close(fd);  
return 0;  
}
```

Compile User App

```
Bash  
gcc app.c -o app
```

Run Application

```
Bash  
./app
```

Example Output:

```
Elapsed Jiffies: 5321  
Elapsed Seconds: 5
```

Flow of Execution

Step 1

Driver loads:

```
C  
start_jiffies = jiffies;
```

Stores current kernel tick count.

Step 2

After some time:

```
Current jiffies = 10500  
Start jiffies   = 5000
```

Step 3

Elapsed time:

```
10500 - 5000 = 5500 jiffies
```

Step 4

Convert to seconds:

```
5500 / HZ
```

If:

```
HZ = 1000
```

then:

```
5 seconds
```

Real-Time Use Cases of Jiffies

1. Driver Timeout Handling

Example:

- Wait for hardware response
- Exit after timeout

```
C
timeout = jiffies + msecs_to_jiffies(1000);
while (!device_ready())
{
    if (time_after(jiffies, timeout))
    {
        printk("Timeout\n");
        break;
    }
}
```

2. Button Debouncing

Ignore repeated button interrupts within small duration.

3. Network Packet Timeout

Used in:

- TCP retransmission
- Packet scheduling
- Connection timeout

4. Polling Hardware Devices

Example:

- UART
- SPI
- I2C device ready checking

5. Measuring Driver Execution Time

```
C
start = jiffies;
/* code */
end = jiffies;
printk("Time = %lu\n", end - start);
```

6. Sleep Operations

```
C
schedule_timeout()
```

internally uses jiffies.

7. Watchdog Timers

Detect hangs or dead devices.

8. Process Scheduling

Kernel scheduler uses timer ticks extensively.

Difference Between Jiffies and Real Time

Jiffies	Real Time Clock
Tick based	Actual wall clock
Kernel internal	Human readable
Fast	More precise
Lightweight	Complex

Important Interview Questions

Why not use delay loops instead of jiffies?

Busy waiting wastes CPU cycles.

Jiffies allow timeout-based efficient waiting.

Why use `time_after()`?

To safely handle overflow.

Is jiffies high precision?

No.

Resolution depends on:

1/HZ

For nanosecond precision:

- `ktime`
- `hrtimers`

are used.

Key Takeaways

- Jiffies = kernel timer tick counter
- Incremented every timer interrupt
- Used for kernel timing operations
- Depends on `HZ`
- Use macros for safe comparison
- Widely used in device drivers and scheduling

Keyboard Interrupt Timeout Using Jiffies

This example demonstrates:

- Capturing keyboard interrupts
- Using **jiffies** for timeout handling
- Detecting inactivity
- Triggering timeout if no key is pressed within a fixed duration

Problem Statement

Create a kernel module that:

- Hooks keyboard IRQ
- Records the last keyboard activity time
- Uses jiffies to detect inactivity
- Prints a timeout message if keyboard is inactive for 5 seconds

Concept

Every key press generates:

IRQ 1

We:

1. Capture keyboard interrupt
2. Store current **jiffies**
3. Periodically check elapsed time
4. Print timeout message if:

Current jiffies - Last key jiffies > 5 seconds

Architecture

```
Keyboard Interrupt
  ↓
Update last_activity_jiffies
  ↓
Kernel Timer Checks Timeout
  ↓
If inactive > 5 sec
  ↓
Print Timeout Message
```

Kernel Module Code

```
C
// keyboard_timeout.c
```

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/interrupt.h>
#include <linux/jiffies.h>
#include <linux/timer.h>

#define KEYBOARD_IRQ 1
#define TIMEOUT_SEC 5

static unsigned long last_activity;
static struct timer_list my_timer;

/* Keyboard Interrupt Handler */
static irqreturn_t keyboard_irq_handler(int irq, void *dev_id)
{
    last_activity = jiffies;

    printk(KERN_INFO "Keyboard Activity Detected\n");

    return IRQ_HANDLED;
}

/* Timer Callback Function */
static void timer_callback(struct timer_list *t)
{
    unsigned long elapsed;

    elapsed = jiffies - last_activity;

    /* Convert timeout seconds into jiffies */
    if (elapsed > secs_to_jiffies(TIMEOUT_SEC))
    {
        printk(KERN_INFO "Keyboard Timeout Occurred\n");

        /* Reset activity */
        last_activity = jiffies;
    }

    /* Restart timer after 1 second */
    mod_timer(&my_timer, jiffies + HZ);
}

/* Module Init */
static int __init keyboard_timeout_init(void)
{
    int result;

    printk(KERN_INFO "Keyboard Timeout Module Loaded\n");

    last_activity = jiffies;

    /* Register Keyboard IRQ */
    result = request_irq(KEYBOARD_IRQ,
                        keyboard_irq_handler,
                        IRQF_SHARED,
                        "keyboard_timeout",
                        (void *)&keyboard_irq_handler);

    if (result)
    {
        printk(KERN_ERR "Cannot register keyboard IRQ\n");
        return result;
    }

    printk(KERN_INFO "Keyboard IRQ Registered\n");

    /* Setup timer */
    timer_setup(&my_timer, timer_callback, 0);

    /* Start timer */
    mod_timer(&my_timer, jiffies + HZ);

    return 0;
}
```

```
/* Module Exit */
static void __exit keyboard_timeout_exit(void)
{
    del_timer(&my_timer);

    free_irq(KEYBOARD_IRQ, (void *)&keyboard_irq_handler);

    printk(KERN_INFO "Keyboard Timeout Module Removed\n");
}

module_init(keyboard_timeout_init);
module_exit(keyboard_timeout_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("OpenAI");
MODULE_DESCRIPTION("Keyboard Interrupt Timeout Using Jiffies");
```

Makefile

```
Makefile
obj-m += keyboard_timeout.o

KDIR = /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

Compile

```
Bash
make
```

Insert Module

```
Bash
sudo insmod keyboard_timeout.ko
```

Check Logs

```
Bash
dmesg -w
```

Expected Output

When pressing keys

```
Keyboard Activity Detected
Keyboard Activity Detected
Keyboard Activity Detected
```

After 5 seconds inactivity

Keyboard Timeout Occurred

Remove Module

```
Bash
sudo rmmod keyboard_timeout
```

Important Concepts Used

Concept	Purpose
<code>request_irq()</code>	Register keyboard interrupt
<code>jiffies</code>	Store time ticks
<code>secs_to_jiffies()</code>	Convert seconds to jiffies
<code>timer_list</code>	Periodic timeout checking
<code>mod_timer()</code>	Restart timer
<code>free_irq()</code>	Release IRQ

How Jiffies Work Here

Suppose:

```
HZ = 1000
```

Then:

```
1 second = 1000 jiffies
```

If last key press occurred at:

```
last_activity = 5000
```

Current:

```
jiffies = 11050
```

Elapsed:

```
11050 - 5000 = 6050 jiffies
```

Timeout check:

```
6050 > 5000
```

Thus:

Keyboard Timeout Occurred

Why Use a Timer Instead of Busy Waiting?

Bad approach:

```
C
while(1)
{
    if(timeout)
}
```

Problems:

- CPU wastage
- System slowdown
- Inefficient kernel behavior

Using kernel timers:

- Event-driven
- Efficient
- Scheduler friendly

Real Use Cases

1. Auto Screen Lock

If keyboard inactive for:

- 1 minute
- 5 minutes

Lock system.

2. Embedded Systems

Detect operator inactivity.

3. Security Systems

Logout idle user automatically.

4. Gaming Devices

Detect controller inactivity.

5. Power Saving

Turn off display after inactivity.

Important Notes

IRQ 1

Standard PC keyboard IRQ:

```
IRQ 1
```

IRQF_SHARED

Keyboard IRQ is shared with other handlers.

Why Use `secs_to_jiffies()`?

Portable across kernels.

Avoid hardcoding:

```
C
5000
```

because HZ may differ.

Important Interview Questions

Why use `jiffies` instead of `sleep()`?

Interrupt handlers cannot sleep.

Jiffies provide non-blocking timing.

Can we use jiffies inside interrupt handler?

Yes.

Very common practice.

Is this hard real-time accurate?

No.

Accuracy depends on:

- Scheduler
 - HZ value
 - Timer resolution
-

Advanced Improvements

You can extend this module to:

- Read actual key scan codes
- Detect specific keys
- Maintain inactivity statistics
- Send signal to user space
- Create `/proc` entry
- Use workqueues/tasklets
- Use high-resolution timers (`hrtimer`)

provide an user app for the above

User Application for Keyboard Timeout Driver

The previous kernel module prints timeout messages using `printk()`.

Now let us create a **user-space application** that:

- Reads timeout events from the driver
- Displays keyboard inactivity notification in user space

For this, we slightly improve the driver:

- Driver exposes a character device
 - User app reads timeout events using `read()`
-

Updated Driver Design

```

Keyboard IRQ
↓
Update jiffies
↓
Timer detects timeout
↓
Store timeout message
↓
User app reads via /dev/kb_timeout

```

Updated Kernel Driver

```

C
// keyboard_timeout.c

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/interrupt.h>
#include <linux/jiffies.h>
#include <linux/timer.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "kb_timeout"
#define KEYBOARD_IRQ 1
#define TIMEOUT_SEC 5

static int major;
static struct timer_list my_timer;
static unsigned long last_activity;

static int timeout_occured = 0;

/* Keyboard IRQ Handler */
static irqreturn_t keyboard_irq_handler(int irq, void *dev_id)
{
    last_activity = jiffies;

    printk(KERN_INFO "Keyboard Activity\n");

    return IRQ_HANDLED;
}

/* Timer Callback */
static void timer_callback(struct timer_list *t)
{
    unsigned long elapsed;

    elapsed = jiffies - last_activity;

    if (elapsed > secs_to_jiffies(TIMEOUT_SEC))
    {
        printk(KERN_INFO "Keyboard Timeout\n");

        timeout_occured = 1;

        last_activity = jiffies;
    }

    mod_timer(&my_timer, jiffies + HZ);
}

/* Read Function */
static ssize_t driver_read(struct file *file,
                           char __user *buf,
                           size_t len,

```

```

                                loff_t *off)
{
    char message[100];
    int bytes;

    if (*off > 0)
        return 0;

    if (timeout_occured)
    {
        bytes = sprintf(message,
                        "Keyboard timeout detected\n");

        timeout_occured = 0;
    }
    else
    {
        bytes = sprintf(message,
                        "No timeout\n");
    }

    copy_to_user(buf, message, bytes);

    *off = bytes;

    return bytes;
}

static struct file_operations fops =
{
    .owner = THIS_MODULE,
    .read = driver_read,
};

static int __init keyboard_timeout_init(void)
{
    int result;

    printk(KERN_INFO "Keyboard Timeout Driver Loaded\n");

    last_activity = jiffies;

    /* Register Char Device */
    major = register_chrdev(0,
                            DEVICE_NAME,
                            &fops);

    printk(KERN_INFO "Major Number = %d\n", major);

    /* Register Keyboard IRQ */
    result = request_irq(KEYBOARD_IRQ,
                        keyboard_irq_handler,
                        IRQF_SHARED,
                        "keyboard_timeout",
                        (void *)&keyboard_irq_handler);

    if (result)
    {
        printk(KERN_ERR "IRQ Registration Failed\n");

        unregister_chrdev(major, DEVICE_NAME);

        return result;
    }

    /* Setup Timer */
    timer_setup(&my_timer, timer_callback, 0);

    mod_timer(&my_timer, jiffies + HZ);

    return 0;
}

static void __exit keyboard_timeout_exit(void)
{

```

```
del_timer(&my_timer);

free_irq(KEYBOARD_IRQ,
        (void *)&keyboard_irq_handler);

unregister_chrdev(major,
                 DEVICE_NAME);

printk(KERN_INFO "Driver Removed\n");
}

module_init(keyboard_timeout_init);
module_exit(keyboard_timeout_exit);

MODULE_LICENSE("GPL");
```

Makefile

```
Makefile
obj-m += keyboard_timeout.o

KDIR = /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

Compile Driver

```
Bash
make
```

Insert Module

```
Bash
sudo insmod keyboard_timeout.ko
```

Check Major Number

```
Bash
dmesg
```

Example:

```
Major Number = 240
```

Create Device File

```
Bash
sudo mknod /dev/kb_timeout c 240 0
```

Give Permission

```
Bash
sudo chmod 666 /dev/kb_timeout
```

User Application

```
C
// user_app.c

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main()
{
    int fd;
    char buffer[100];

    fd = open("/dev/kb_timeout", O_RDONLY);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    while (1)
    {
        memset(buffer, 0, sizeof(buffer));

        lseek(fd, 0, SEEK_SET);

        read(fd, buffer, sizeof(buffer));

        printf("%s", buffer);

        sleep(1);
    }

    close(fd);

    return 0;
}
```

Compile User Application

```
Bash
gcc user_app.c -o user_app
```

Run User Application

```
Bash
./user_app
```

Expected Output

While Typing

```
No timeout
No timeout
No timeout
```

Stop Typing for 5 Seconds

```
Keyboard timeout detected
```

Execution Flow

Step 1

Key press interrupt occurs:

```
C
last_activity = jiffies;
```

Step 2

Timer checks inactivity every second:

```
C
elapsed = jiffies - last_activity;
```

Step 3

If elapsed exceeds timeout:

```
C
elapsed > secs_to_jiffies(5)
```

then:

```
C
timeout_occured = 1;
```

Step 4

User application reads:

```
Keyboard timeout detected
```

Real Driver Enhancements

Production drivers may additionally use:

- `poll()`
- `select()`
- `wait queues`
- `epoll`
- `netlink sockets`
- `ioctl`
- `procfs`
- `sysfs`

instead of repeated polling.

Better Design (Interview Point)

Current user app uses:

```
C  
sleep(1)
```

This is polling.

Better Linux design:

- `wait queue`
- `blocking read`
- `async notification`

This avoids CPU wastage.

Interview Questions

Why use timer instead of checking inside IRQ?

Interrupt handlers must be short and fast.

Timeout logic should run outside interrupt context.

Why use character device?

Allows user-space communication.

Why use jiffies here?

Efficient lightweight kernel timing.

Next Advanced Examples You Can Build

1. Wait Queue + Blocking Read
2. Poll/select Driver
3. Netlink Notification
4. Input Subsystem Driver
5. Tasklet/Workqueue Version
6. High Resolution Timer Version
7. Keyboard Scan Code Logger
8. `/proc` Interface Version
9. Sysfs Timeout Control
10. Interrupt Debounce Driver

extern unsigned long volatile jiffies what is extternn adn volatile ehre

Understanding This Line

```
C
extern unsigned long volatile jiffies;
```

This declaration contains 3 important concepts:

1. `extern`
2. `unsigned long`
3. `volatile`

Let us understand each clearly.

Full Breakdown

```
C
extern unsigned long volatile jiffies;
```

means:

"There exists a global variable named `jiffies` of type `unsigned long`, whose value can change anytime unexpectedly."

1. What is extern?

Meaning

`extern` tells the compiler:

"This variable is defined somewhere else."

It is only a declaration, NOT memory allocation.

Example Without extern

file1.c

```
C
int count = 10;
```

Memory is allocated here.

file2.c

```
C
extern int count;
```

No new memory allocated.

It simply says:

"Use the variable defined in another file."

Why extern for jiffies?

Because `jiffies` is actually defined inside kernel source code.

Somewhere in kernel:

```
C
unsigned long volatile jiffies;
```

Your driver only accesses it using:


```
C
extern unsigned long volatile jiffies;
```

Analogy

Imagine:

```
Kernel owns the house (memory)
Driver only knows the address
```

extern means:

```
"The variable exists elsewhere."
```

If extern is NOT used?

Compiler assumes:

```
C
unsigned long volatile jiffies;
```

means:

```
Create a NEW variable
```

This causes:

- multiple definition error
- linker error

2. What is unsigned long?

It is the datatype.

unsigned

Stores only positive values.

Range (32-bit):

```
0 to 4294967295
```

long

Typically:

- 4 bytes on 32-bit
- 8 bytes on 64-bit

Kernel uses large counters for timing.

Why unsigned long for jiffies?

Because jiffies continuously increase:

```
0 → 1 → 2 → 3 → ...
```

Need large positive range.

3. What is volatile?

This is VERY important in embedded/Linux driver interviews.

Meaning of volatile

volatile tells compiler:

```
"This variable can change anytime unexpectedly."
```

So:

```
DO NOT optimize accesses to it.  
Always read from memory.
```

Why Needed for jiffies?

Because jiffies changes due to:

```
Timer Interrupts
```

NOT by normal program flow.

Hardware timer interrupt updates jiffies asynchronously.

Without volatile (Danger)

Suppose:

```
C  
while (jiffies < 1000)  
{  
}
```

Compiler may optimize as:

```
jiffies never changes inside loop
```

and convert to:

```
C
if (jiffies < 1000)
{
    while(1);
}
```

Infinite loop problem.

With volatile

Compiler MUST reload jiffies from memory every time.

Correct behavior:

```
Read actual updated value
```

Real Embedded Example

Without volatile

```
C
int flag = 0;
while(flag == 0)
{
}
```

If interrupt changes `flag`,
compiler may optimize loop forever.

Correct Version

```
C
volatile int flag = 0;
```

Now compiler checks memory repeatedly.

In Kernel Context

`jiffies` changes from:

Hardware timer interrupt

So kernel declares:

```
C
volatile
```

to prevent wrong optimization.

Important Interview Question

Does volatile make code thread-safe?

NO.

volatile only prevents compiler optimization.

It does NOT provide:

- mutex
- locking
- atomicity

Volatile vs Atomic

Feature	volatile	atomic
Prevent optimization	YES	YES
Thread safe	NO	YES
Interrupt safe	NO	YES
Synchronization	NO	YES

Real Kernel Definition

Linux internally defines:

```
C
extern unsigned long volatile jiffies;
```

and also:

```
C
extern u64 jiffies_64;
```

for 64-bit timekeeping.

Memory View

```
RAM
-----
Address 0x1000 → jiffies = 5000
-----

Timer interrupt:
5000 → 5001 → 5002
```

Your driver always reads updated RAM value because of **volatile**.

Simple Demonstration Program

Without volatile

```
C
#include <stdio.h>

int flag = 0;

int main()
{
    while(flag == 0)
    {
    }

    printf("Done\n");
}
```

Compiler may optimize infinitely.

With volatile

```
C
#include <stdio.h>

volatile int flag = 0;

int main()
{
    while(flag == 0)
    {
    }

    printf("Done\n");
}
```

Works correctly when another thread/interrupt changes **flag**.

Summary

Keyword	Meaning
extern	Variable defined elsewhere
unsigned long	Large positive integer
volatile	Variable may change unexpectedly

Final Meaning of Entire Line

```
C
extern unsigned long volatile jiffies;
```

means:

```
"There is a global kernel variable named jiffies,
defined elsewhere,
which stores timer ticks,
and whose value may change asynchronously,
so compiler must always read actual memory."
```

Most Important Interview Point

volatile is heavily used in:

- Device drivers
- Memory mapped registers
- Interrupt flags
- Hardware status registers
- Shared ISR variables
- DMA buffers
- RTOS development
- Embedded firmware

Very common interview topic in:

- Linux kernel
- Embedded systems
- Firmware
- Device driver roles